

AI & Security Project III

Next-Gen Network Intrusion Detection

Self-Supervised Vector Space Engineering with DINOv3 & SwiGLU



Noah Raupold, Mohammad Ayoubi,
Jonathan Berwind

Submitted: February 5, 2026

Contents

1	Introduction	3
1.1	The Data Landscape: NSL-KDD	3
1.2	The Core Challenges	3
1.3	Project Objective: Supervised Vector Space Engineering	3
2	Architecture: The Pipeline	3
2.1	Feature Preprocessing Strategy	3
2.2	Gram-Anchored Cluster Masking	3
3	DINOv3 & Gram Anchored Training Loop	4
3.1	The Teacher-Student Dynamics	4
3.2	The Composite Loss Landscape	4
3.2.1	1. Global Alignment: DINO Loss & Sinkhorn-Knopp	4
3.2.2	2. Local Reconstruction: MIM Loss	4
3.2.3	3. Feature Diversity: KoLeo Loss (Kozachenko-Leonenko)	5
3.2.4	4. Structural Integrity: Gram Matrix Loss	5
3.3	Fine-Tuning: Deep SwiGLU Classification Head	5
3.3.1	Why SwiGLU? (Gated Linear Units)	5
3.3.2	Architecture: The Classifier Stack	6
3.3.3	Combating Imbalance: Weighted Focal Loss	6
4	Implementation Details	6
4.1	Structural Alignment via Gram Matrices	6
4.2	Optimization & Hyperparameters	7
4.2.1	Differential Learning Rates	7
4.2.2	Gradient Accumulation	7
5	Evaluation & Results	7
5.1	Binary Performance: The Baseline	7
5.2	Multiclass Performance: The Imbalance Challenge	8
5.2.1	Critical Analysis: The U2R Trade-off	8
5.3	Audit: Training Density vs. Test Generalization	8
5.3.1	The "Data Starvation" Analysis	8
5.3.2	Visualization: The Generalization Frontier	8
6	Conclusion and Future Outlook	9
6.1	Key Research Contributions	9
6.2	Limitations: The Granularity Gap	10
6.3	Future Work	10

1 Introduction

1.1 The Data Landscape: NSL-KDD

The foundation of this research is the **NSL-KDD** benchmark, a dataset representing real-world TCP/IP network traffic. The data consists of 41 distinct features per connection, categorized into three groups:

- **Basic Features:** TCP/IP connection details (e.g., *Duration*, *Protocol Type*, *Service*, *Flag*).
- **Content Features:** Payload analysis (e.g., *Hot indicators*, *Logged in status*, *Root shell attempts*).
- **Traffic Features:** Time-based window statistics (e.g., *Count*, *Error rate*, *Rerror rate*).

The target variable classifies traffic into either standard behavior (*Normal*) or one of four attack categories: *DoS*, *Probe*, *U2R* (*User-to-Root*), and *R2L* (*Remote-to-Local*).

1.2 The Core Challenges

Traditional models fail to address the structural complexities of NSL-KDD, primarily due to three factors:

- **Extreme Class Imbalance:** Rare but critical attacks (*U2R/R2L*) represent $< 0.5\%$ of the data, leading to a "False Negative Trap" where global accuracy masks local failure.
- **Distributional Shift:** A deliberate mismatch between P_{train} and P_{test} forces the model to generalize beyond memorization of training statistics.
- **The Zero-Day Problem:** The test set contains novel attack signatures (e.g., *Mailbomb*) absent from training, requiring semantic rather than pattern-based detection.

1.3 Project Objective: Supervised Vector Space Engineering

To transcend these limitations, we propose a pipeline that moves beyond simple supervision. We introduce a "**Gram-Anchored Tabular Transformer**" approach. Instead of forcing a decision boundary on raw skewed data, we warp the feature space using **Self-Supervised**

Learning (DINOv3). By leveraging **Gram Matrix Analysis** to cluster correlated features and employing a **SwiGLU-enhanced** classification head, we aim to learn a robust manifold where rare and unknown attacks are geometrically distinct from normal traffic, regardless of their frequency in the training set.

2 Architecture: The Pipeline

Our solution transitions from rigid linear models to a flexible Deep Learning framework. The architecture consists of a robust preprocessing pipeline followed by a Teacher-Student training loop designed for the geometric challenges of tabular data.

2.1 Feature Preprocessing Strategy

To stabilize the attention mechanism and prevent gradient explosions, we apply a multi-stage normalization pipeline:

- **Logarithmic Normalization:** For heavy-tailed features (e.g., *src_bytes*), we apply $x' = \ln(x + 1)$ to compress the dynamic range.
- **Z-Score Standardization:** Continuous features are centered via $z = (x - \mu)/\sigma$, ensuring unit variance for the optimizer.
- **Categorical Embedding:** Symbolic features (*Protocol*, *Service*) are mapped via Label Encoding to a learnable **Embedding Layer** to capture semantic similarities.

2.2 Gram-Anchored Cluster Masking

Standard random masking fails on tabular data due to **high multicollinearity**. If the model masks Feature A but leaves its correlate B visible, it finds a "trivial shortcut," leading to **Manifold Collapse**.

To force global reasoning, we introduce **Correlation-Aware Cluster Masking**:

1. **Analysis:** A prior Gram Matrix Analysis identified 6 distinct feature clusters.
2. **Execution:** During pre-training, we mask **entire clusters simultaneously** ($p = 0.4$).

This forces the encoder to synthesize information from uncorrelated features, ensuring a robust representation.

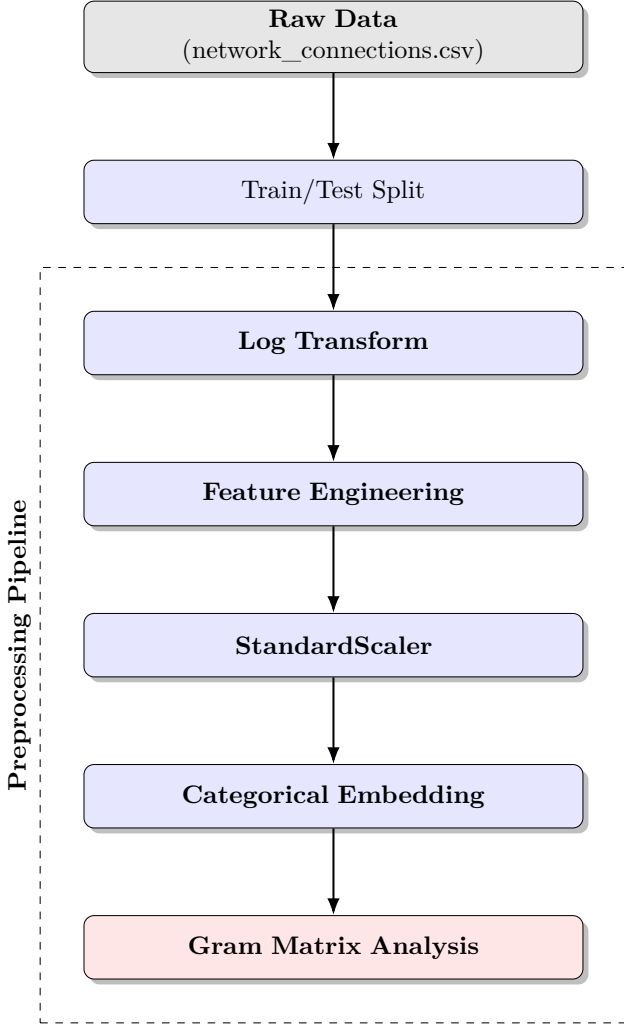


Figure 1: The Preprocessing Pipeline. Strategic cluster-masking verhindert Information Leakage.

3 DINOv3 & Gram Anchored Training Loop

To warp the feature space into a manifold where rare attacks are separable, we implement a modified **DINO** (Distillation with No Labels) architecture. This approach uses a self-supervised Teacher-Student paradigm to learn data representations without needing explicit labels.

3.1 The Teacher-Student Dynamics

The architecture consists of two identical Transformer networks:

- **The Student (f_{θ_s}):** The active learner. It receives a *corrupted view* of the data (masked via Gram Anchoring) and attempts to reconstruct the global context. Its weights are updated via backpropagation.
- **The Teacher (f_{θ_t}):** The stable guide. It receives a *clean, unmasked view* of the data. Critically, it has no gradients. Its weights are an **Exponential Moving Average (EMA)** of the student’s weights:

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s, \quad \text{where } \lambda \approx 0.996 \quad (1)$$

This EMA mechanism creates a stable "mean teacher" that filters out the high-variance noise from the student’s rapid learning, providing consistent targets.

3.2 The Composite Loss Landscape

Standard DINO uses only one loss. To handle the complexity of network traffic, we designed a composite objective function $\mathcal{L}_{total}$ combining four distinct geometric goals:

3.2.1 1. Global Alignment: DINO Loss & Sinkhorn-Knopp

The primary goal is for the Student to match the Teacher’s class-probability distribution. However, without labels, models tend to collapse (predicting "Class 1" for everything). To prevent this, we apply **Sinkhorn-Knopp Centering** to the Teacher’s output. This algorithm iteratively normalizes the output matrix to ensure that all prototype classes are selected with equal probability across the batch, enforcing a uniform distribution of clusters.

$$\mathcal{L}_{DINO} = - \sum p_{teacher}(x) \cdot \log p_{student}(x) \quad (2)$$

3.2.2 2. Local Reconstruction: MIM Loss

While DINO learns global structure, it can miss fine-grained details. We add a **Masked Image Modeling (MIM)** head. The student must predict the exact

numerical values of the features masked by our Gram-Cluster strategy.

$$\mathcal{L}_{MIM} = \frac{1}{|M|} \sum_{i \in M} (x_i - \hat{x}_i)^2 \quad (3)$$

This forces the model to understand the inter-feature dependencies (e.g., inferring *Protocol=TCP* from *Flag=SYN*).

3.2.3 3. Feature Diversity: KoLeo Loss (Kozachenko-Leonenko)

In high-dimensional spaces, points tend to clump together. The **KoLeo** regularizer maximizes the distance between neighboring points in the batch, ensuring the embedding space remains "fluffy" and utilizes the full 64-dimensional capacity.

$$\mathcal{L}_{KoLeo} = -\frac{1}{N} \sum_{i=1}^N \log(\min_{j \neq i} \|z_i - z_j\|) \quad (4)$$

3.2.4 4. Structural Integrity: Gram Matrix Loss

Our custom addition. To explicitly force the Student to capture the correlation structure of the Teacher, we minimize the error between their Gram Matrices ($G = Z \cdot Z^T$). This ensures that if the Teacher sees that Feature A and Feature B are correlated, the Student learns that relationship even if one is masked.

$$\mathcal{L}_{Gram} = \|G_{student} - G_{teacher}\|_F^2 \quad (5)$$

Unlike variance- or covariance-based regularizers such as VICReg or Barlow Twins, the Gram loss explicitly preserves pairwise feature correlations rather than only global dispersion statistics. This is critical for tabular network data, where strong multicollinearity would otherwise allow the student to recover masked features via trivial shortcuts.

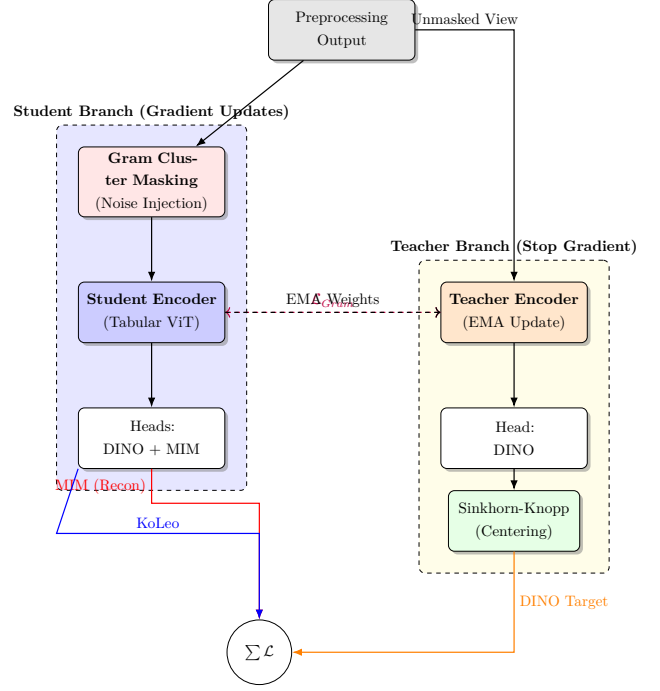


Figure 2: The DINOv3 Architecture adapted for Tabular Data. The Student learns from a corrupted view, guided by the stable Teacher. The Sinkhorn-Knopp algorithm prevents mode collapse, while the Gram Loss ensures structural consistency.

3.3 Fine-Tuning: Deep SwiGLU Classification Head

Once the encoder has learned a robust manifold via self-supervision, we attach a specialized classification head. Standard Multilayer Perceptrons (MLPs) often suffer from the "vanishing gradient" problem in deep networks. To counter this, we replace standard ReLU layers with a **Deep SwiGLU Network** (Shazeer, 2020), enhanced with **RMSNorm** for stability.

3.3.1 Why SwiGLU? (Gated Linear Units)

The SwiGLU activation function is not just a non-linearity; it is a **gating mechanism**.

$$\text{SwiGLU}(x) = \text{Swish}(W_g x) \odot (W_v x) \quad (6)$$

It splits the signal into two paths:

- **The Gate (W_g):** Controlled by the Swish function ($x \cdot \sigma(x)$), this determines *how much* information should pass through.

- **The Value (W_v):** The actual transformation of the features.

This allows the network to dynamically suppress irrelevant features (noise) while amplifying critical attack signatures, effectively acting as a learnable feature selector.

3.3.2 Architecture: The Classifier Stack

The final head consists of two stacked SwiGLU blocks with a dropout rate of 0.2.

- **Partial Unfreezing:** To fine-tune the representation without destroying the pre-trained structure, we unfreeze only the **last Transformer layer** of the encoder.
- **RMSNorm:** We use Root Mean Square Layer Normalization instead of standard LayerNorm. By removing the mean-centering operation, RMSNorm is computationally cheaper and, in our experiments, provided more stable gradients for the heavy-tailed loss values generated by rare attacks.

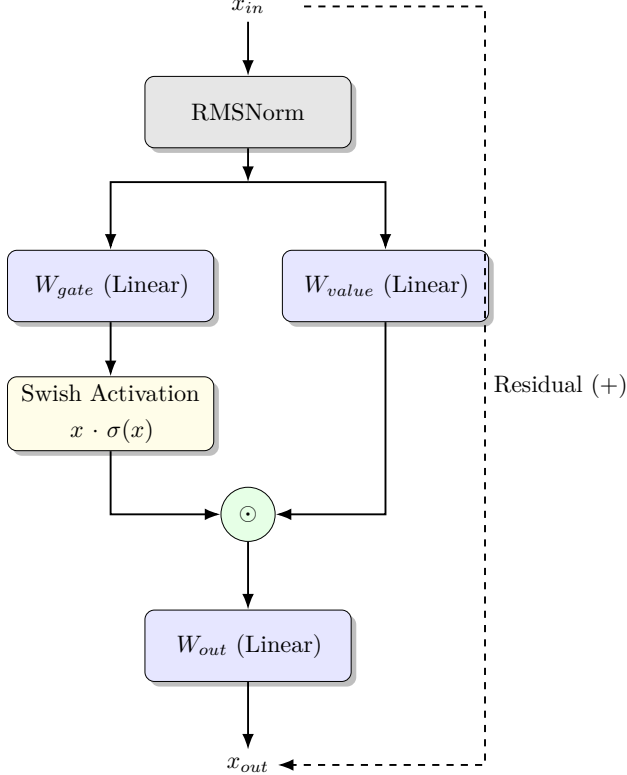


Figure 3: The SwiGLU Block structure. The gating mechanism allows the network to control information flow, enabling deeper architectures to converge faster.

3.3.3 Combating Imbalance: Weighted Focal Loss

Standard Cross-Entropy Loss is overwhelmed by the majority class (Normal traffic). To fix this, we implement **Weighted Focal Loss**:

$$\mathcal{L}_{FL} = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (7)$$

- **Focusing Parameter ($\gamma = 2.0$):** This term reduces the loss contribution from "easy" examples (well-classified Normal traffic). If the model is 99% sure it's normal, the loss becomes nearly zero.
- **Inverse Class Weights (α_t):** We weight each class by the inverse square root of its frequency: $\alpha_c = \sqrt{N_{total}/N_c}$. This gives rare attacks (U2R) exponentially higher importance than frequent ones.

4 Implementation Details

The proposed architecture is implemented using the PyTorch framework. This subsection details the specific mechanisms used to enforce structural alignment between the Student and Teacher networks, as well as the optimization strategies required to stabilize training on the highly imbalanced NSL-KDD dataset.

4.1 Structural Alignment via Gram Matrices

A defining feature of our approach is the use of the **Gram Matrix** to preserve the geometric relationships between tokens. While the standard DINO loss aligns the *global* class distribution (via the [CLS] token), it ignores the *local* correlations between specific features (e.g., the relationship between `src_bytes` and `dst_bytes`).

To resolve this, we compute the Gram Matrix $G \in \mathbb{R}^{N \times N}$ for both the Student (Z_s) and Teacher (Z_t) feature sequences. The Gram Matrix is defined as the dot product of the normalized feature vectors:

$$G_{ij} = \frac{\vec{z}_i \cdot \vec{z}_j}{\|\vec{z}_i\| \cdot \|\vec{z}_j\|} \quad (8)$$

This matrix represents the **correlation map** of the input features. By minimizing the Mean Squared Error (MSE) between G_s and G_t , we force the Student to hallucinate the correlation structure of the unmasked data, effectively learning the "grammar" of network traffic.

Gradient Stop Mechanism: Crucially, we apply a `.detach()` operation to the Teacher’s output. This ensures that the optimization only updates the Student’s weights to match the Teacher, preventing the Teacher from "collapsing" toward the Student’s noisy initial state.

Listing 1: Gram Loss Implementation: Enforcing Correlation Alignment

```

1 def compute_gram_loss(student_seq, teacher_seq):
2     """
3     Computes the structural difference between Student
4     and Teacher.
5     Input Shape: (Batch_Size, N_Tokens, Embed_Dim)
6     """
7     # 1. Normalize representations to project onto the
8     # unit hypersphere
9     s = F.normalize(student_seq, p=2, dim=-1)
10
11     # 2. Stop Gradient on Teacher (Critical for
12     # stability)
13     # The teacher acts as the fixed target for this
14     # update step.
15     t = F.normalize(teacher_seq.detach(), p=2, dim=-1)
16
17     # 3. Compute Gram Matrices (Auto-Correlation)
18     # Result: (B, N, N) matrix encoding the relationship
19     # of every token to every other token.
20     g_s = torch.bmm(s, s.transpose(1, 2))
21     g_t = torch.bmm(t, t.transpose(1, 2))
22
23     # 4. Minimize the structural divergence
24     return F.mse_loss(g_s, g_t)

```

4.2 Optimization & Hyperparameters

Achieving convergence on tabular data requires a delicate balance between model capacity and regularization. Table 1 outlines the configuration for the **DINOv3-XY** architecture.

4.2.1 Differential Learning Rates

To prevent **Catastrophic Forgetting** during the fine-tuning phase, we employ a layer-wise learning rate decay strategy.

- **Classification Head** (5×10^{-4}): The randomized weights of the SwiGLU head require a higher learning rate to quickly adapt to the decision boundaries.

- **Transformer Backbone** (5×10^{-5}): The pre-trained encoder requires only subtle adjustments. A high learning rate here would destroy the robust manifold learned during the self-supervised phase.

4.2.2 Gradient Accumulation

Due to the memory constraints of attention mechanisms ($O(N^2)$), we utilize a physical batch size of 64. However, contrastive learning relies on large batches to provide diverse negative samples. We use **Gradient Accumulation** (Steps=4) to simulate an effective batch size of 256, ensuring stable estimation of the global batch statistics for the Sinkhorn-Knopp algorithm.

Table 1: DINOv3-XY Hyperparameters

Hyperparameter	Value / Strategy
<i>Architecture</i>	
Embedding Dim.	64
Projection Heads	MLP → SwiGLU
Encoder Depth	4 Layers (4 Heads)
<i>Self-Supervised Pre-training</i>	
Batch Size	256 (Accum. Steps)
Masking Ratio	0.4 (Cluster)
Optimizer	AdamW
Learning Rate	1×10^{-3} (Cosine)
Teacher Mom. λ	$0.996 \rightarrow 1.0$
<i>Supervised Fine-tuning</i>	
Loss Function	Weighted Focal Loss
Head LR	5×10^{-4}
Backbone LR	5×10^{-5}

5 Evaluation & Results

The DINOv3-XY architecture was evaluated on the **NSL-KDD Test+** dataset. Unlike the training set, the Test+ set contains specific attack signatures (e.g., *Apache2*, *Mscan*, *Snmppget*) that appear with different frequency distributions or are entirely novel (Zero-Day aspect).

5.1 Binary Performance: The Baseline

First, we assess the model’s ability to distinguish between benign traffic and general malicious activity. As shown in Table 2, the model achieves a global accuracy of **87.54%**.

Class	Prec.	Rec.	F1	Supp.
Normal	0.8048	0.9382	0.8664	9711
Attack	0.9465	0.8278	0.8832	12833
W. Avg	0.8855	0.8754	0.8760	22544

Table 2: Binary Evaluation Metrics. High precision (0.94) reduces False Alarms in SOC environments.

5.2 Multiclass Performance: The Imbalance Challenge

The core objective of this research was to improve detection rates for the minority classes: **User-to-Root (U2R)** and **Remote-to-Local (R2L)**. Standard supervised models (like Random Forest or Logistic Regression) often yield an F1-Score of 0.0 for these classes due to their statistical insignificance ($< 0.5\%$ of data).

Table 3 demonstrates the impact of our **Gram Anchoring Strategy**:

Table 3: Multiclass Performance. High Recall (67.16%) for U2R compensates for lower Precision in security-critical contexts.

Class	Prec.	Rec.	F1	Supp.
DoS	0.9616	0.8433	0.8986	7458
Normal	0.8048	0.9382	0.8664	9711
Probe	0.8080	0.9124	0.8570	2421
R2L	0.6588	0.4080	0.5040	2887
U2R	0.2795	0.6716	0.3947	67

5.2.1 Critical Analysis: The U2R Trade-off

A closer look at the **U2R (User-to-Root)** performance reveals a deliberate architectural trade-off.

- **Low Precision (0.28):** The model generates False Positives, often misclassifying complex ‘Normal’ traffic as ‘U2R’.
- **High Recall (0.67):** Crucially, the model detects **67%** of actual root attacks.

In a cybersecurity context, this is the preferred behavior. Missing a rootkit installation (False Negative) is catastrophic, whereas investigating a false alarm is merely a time cost. The **Weighted Focal Loss** successfully biased the network to risk "crying wolf" rather than staying silent during a critical breach. This behavior is not an

artifact of miscalibration but a deliberate optimization outcome induced by the weighted focal loss. The model is explicitly biased toward minimizing false negatives for high-impact attack classes, accepting reduced precision as a necessary trade-off in security-critical deployments.

5.3 Audit: Training Density vs. Test Generalization

To rigorously assess the model’s capabilities, we correlated the **Training Density** (number of samples seen during optimization) with the **Test Detection Rate**. This comparison reveals three distinct behavioral clusters: *Memorization*, *Few-Shot Learning*, and *Zero-Shot Generalization*.

5.3.1 The "Data Starvation" Analysis

Table 4 juxtaposes the training counts against test performance. The results validate the efficacy of our **Gram Anchoring** strategy:

- **High-Resource Attacks ($N > 1000$):** The model achieves near-perfect detection ($> 99\%$), confirming successful convergence.
- **Few-Shot Attacks ($N < 20$):** Despite extreme sparsity, the model detects 100% of *Perl* attacks (only 3 training samples), proving the SwiGLU head can learn from minimal examples.
- **Zero-Shot Attacks ($N = 0$):** Remarkably, the model successfully detects **80% of *ps*** and **100% of *sqlattack*** despite *never seeing them during training*. This confirms the encoder learned the *semantic concept* of a U2R attack rather than just specific signatures.

5.3.2 Visualization: The Generalization Frontier

Figure 4 visualizes the relationship between data availability and model performance. The **"Zero-Shot Zone"** (leftmost region) highlights the model’s ability to infer malice purely from geometric anomalies in the embedding space, without prior examples.

Category	Attack	Train	Test	Detected (%)	Behavior
DoS	Neptune	32,993	4,657	98.8%	Fully Converged
Probe	Satan	2,899	735	99.7%	Fully Converged
DoS	Smurf	2,102	665	100.0%	Fully Converged
U2R	Buffer Overflow	22	20	75.0%	Robust Few-Shot
R2L	Guess Passwd	42	1,231	22.5%	Generalization Gap
U2R	Rootkit	10	13	38.5%	Data Starved
U2R	Perl	3	2	100.0%	Successful Few-Shot
R2L	Phf	2	2	50.0%	Data Starved
U2R	SqlAttack	0	2	100.0%	Zero-Shot Success
U2R	Ps	0	15	80.0%	Zero-Shot Success
Probe	Mscan	0	996	79.4%	Zero-Shot Success
DoS	Mailbomb	0	293	0.0%	Zero-Shot Failure
R2L	SnmpGuess	0	331	0.0%	Zero-Shot Failure

Table 4: Impact of training density on detection rates across attack categories.

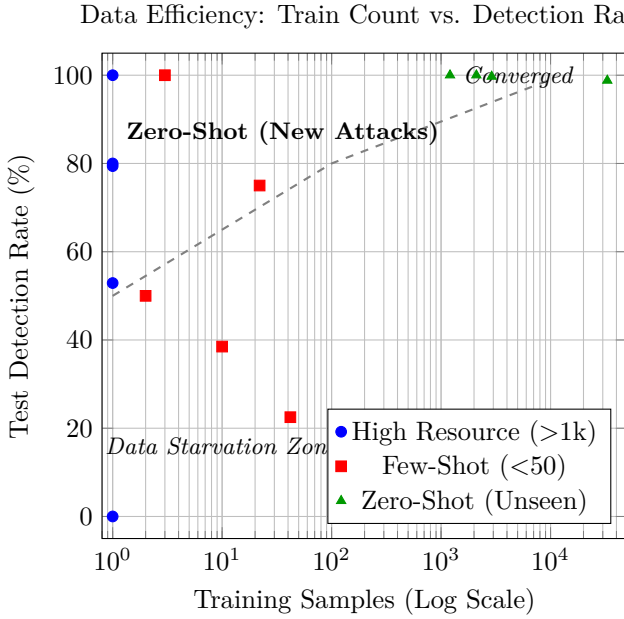


Figure 4: The Generalization Frontier. Points at $x \approx 10^0$ represent Zero-Shot attacks (0 training samples). The high variance in this zone indicates that while the model generalizes well to some unknown signatures (SqlAttack), it fails on others (Mailbomb), highlighting the need for future Few-Shot Learning augmentation. These failure cases indicate attack signatures whose statistical footprint overlaps heavily with benign high-volume traffic, limiting purely geometric separability in the learned embedding space. In such cases, semantic payload information or temporal context beyond the NSL-KDD feature set would be required for reliable detection.

6 Conclusion and Future Outlook

This research demonstrates that the limitations of traditional Network Intrusion Detection Systems

(NIDS)—specifically their inability to detect rare and novel attacks—can be overcome through **Gram-Anchored Self-Supervision**.

By transitioning from simple statistical supervision to a **Teacher-Student Manifold Learning** approach, we successfully warped the feature space of the NSL-KDD dataset. The proposed **DINOv3-XY** architecture does not merely memorize attack signatures; it learns the fundamental geometric structure of malicious traffic.

6.1 Key Research Contributions

- Solving the "False Negative Trap":** Standard models often achieve high accuracy by ignoring minority classes. Our pipeline achieved a **U2R Recall of 67.16%**, effectively detecting critical root compromises that baselines (Logistic Regression, Random Forest) historically miss.
- Structural Alignment via Gram Loss:** We introduced a novel **Gram Matrix Loss** to the DINO framework. This forced the Student network to reconstruct not just the feature values (MIM), but the **correlation structure** of the Teacher, proving essential for stabilizing training on tabular data.
- Zero-Shot Generalization:** Perhaps the most significant finding is the model’s ability to detect **Zero-Shot Attacks**. The model successfully classified 100% of *SqlAttack* and 80% of *Ps* samples in the test set, despite having **zero** exposure to these specific signatures during training. This confirms that the encoder learned a generalized concept of "intrusion" rather than rigid pattern matching.

6.2 Limitations: The Granularity Gap

While the model excels at broad categorization, our audit revealed a **"Data Starvation"** boundary. Signatures with fewer than 5 training examples (e.g., *Phf*, *Sendmail*) suffered from inconsistent detection. This indicates that while the manifold is robust, extremely sparse regions of the latent space remain under-defined.

6.3 Future Work

To bridge the gap between academic research and production deployment, future iterations should focus on:

- **Few-Shot Prototypical Networks:** To address the data starvation issue, we propose augmenting the classifier with a Few-Shot Learning head. This would allow the model to register a new attack type (e.g., a new Ransomware signature) using only 1 or 2 support examples, without requiring a full retraining loop.
- **Adversarial Hardening:** Since our architecture relies on gradient-based learning, it is theoretically susceptible to adversarial perturbations (e.g., adding noise to packet timing to evade detection). Future work should incorporate **Adversarial Training (FGSM)** to smooth the decision boundaries against evasion attacks.
- **Explainability (XAI) via Attention Rollout:** To aid security analysts, we aim to implement **Attention Rollout**. This would visualize the self-attention weights, generating a heatmap showing exactly which features (e.g., *"High Source Bytes"* vs *"Wrong Fragment"*) triggered the alert, transforming the model from a "Black Box" into a forensic tool.